FIN42110 - Financial Data Science

Detecting Informed Trading on Polymarket: A Geopolitical Prediction Market Analysis

April 2026

UCD Michael Smurfit Graduate Business School

Polymarket

johnstew324/polymarket-whale-watch

Student Name 1	Oisín Moffatt	**Student Number 1**	21510573
Student Name 2	John Stewart	**Student Number 2**	25210841
Student Name 3	Adam Ajmal	**Student Number 3**	20309543
Module Title	Financial Data Science for Trading & Risk Management	**Module Co-ordinator**	Dr Richard McGee

Procedures for Submission and Late Submission: ensure that you have checked the School's procedures for the submission of assessments. Note: There are penalties for the late submission of assessments. For further information please see the University's Policy on Late Submission of Coursework, http://www.ucd.ie/registrar/.

Plagiarism: the unacknowledged inclusion of another person's writings or ideas or works, in any formally presented work (including essays, examinations, projects, laboratory reports or presentations). The penalties associated with plagiarism are designed to impose sanctions that reflect the seriousness of the University's commitment to academic integrity. Ensure that you have read the University's Briefing for Students on Academic Integrity and Plagiarism and the UCD Plagiarism Statement, Plagiarism Policy and Procedures, http://www.ucd.ie/registrar/.

Declaration of Authorship

We declare that all material in this assessment is our own work except where there is clear acknowledgement and appropriate reference to the work of others.

Signed ..... Oisín Moffatt .....

Signed ..... John Stewart .....

Signed ..... Adam Ajmal .....

Signed ..... Andrew Holton .....

Contents

0.1 Data Source Evaluation 4
0.2 Pipeline Architecture 5
0.3 Database Schema 10

A Data Download **13**

B Pipeline Source Code **13**

C Pipeline Source Code **13**

0.1 Data Source Evaluation

Polymarket is a decentralised prediction market built on Polygon, a proof-of-stake Ethereum sidechain [Polymarket(2025)]. Users can trade on the outcomes of real-world events using USDC as collateral. The technical structure of these markets is defined by:

- **Market Mechanics:** Each market generates a pair of binary outcome tokens `YES`/`NO`. These outcomes are priced between $0 and $1, which directly reflects the market's implied probability of these event occurring.
- **Token Standards:** Each binary outcome is implemented as **ERC-1155** assets - fungible tokens with 6-decimal precision, traded in discrete units.
- **Settlement and Transparency:** All trades are settled through the CTF Exchange smart contract and are permanently recorded on-chain as `OrderFilled` events.
- **Data Accessibility:** both counterparty wallet addresses are therefore publicly visible in every event, making the full trade history queryable directly from the blockchain.

Trade History Acquisition:

Obtaining the history at wallet level and at scale required evaluating many sources of data. Table 1 summarises the sources evaluated and the reason each was rejected.

Table 1: Data sources evaluated and rejected.

Source	Reason rejected
Polymarket Data API `/data/trades`	Current live data but hard capped at 3,100 trades per market. 426 markets exceeded this threshold. These are the highest-volume geopolitical markets, precisely the most relevant for analysis.
Dune Analytics	Full history available in principle but free tier capped at ∼400,000 rows with complete access paywalled.
Allium `predictions.allium.so`	Not publicly accessible, requires enterprise demo approval.
CryptoHouse `clickhouse.com`	No Polymarket data, Ethereum and Solana only.
Goldsky `polymarket-orderbook-resync/prod`	A new subgraph that is actively indexed but does not include maker and taker wallet addresses entirely.

Chosen source: S3 snapshot of on-chain events. The complete `OrderFilled` event history was sourced from a community-maintained S3 archive published by `warproxxx/poly_data` [warproxxx(2025)]. The pre-built snapshot was used in place of scraping the live subgraph directly. Scraping the 2023-2025 window from scratch via GraphQL pagination takes over two days (as documented in the github repository). Validating that the modeling approach gives meaningful results before investing

in a full live pipeline, was the best approach. Extending the dataset beyond October 2025 and creating a live pipeline is a possible future improvement. Table 2 summarises the resulting dataset.

Table 2: S3 snapshot properties.

Property	**Value**
File	`orderFilled_complete.csv.xz`
Compressed size	5.9 GB
Uncompressed size	∼35–40 GB
Total rows	150M+
Date range	1 February 2023 - 7 October 2025
Hosted at	`polydata-archive.s3.us-east-1.amazonaws.com`

0.2 Pipeline Architecture

The data collection pipeline consists of a snapshot download followed by five Python modules executed sequentially, with each step's output feeding directly into the next. Figure 1 shows the full flow from the S3 archive and external APIs through to the analytical DuckDB database. Full setup and reproduction instructions are provided in the project `README.md`[1]

[1] johnstew324/polymarket-whale-watch

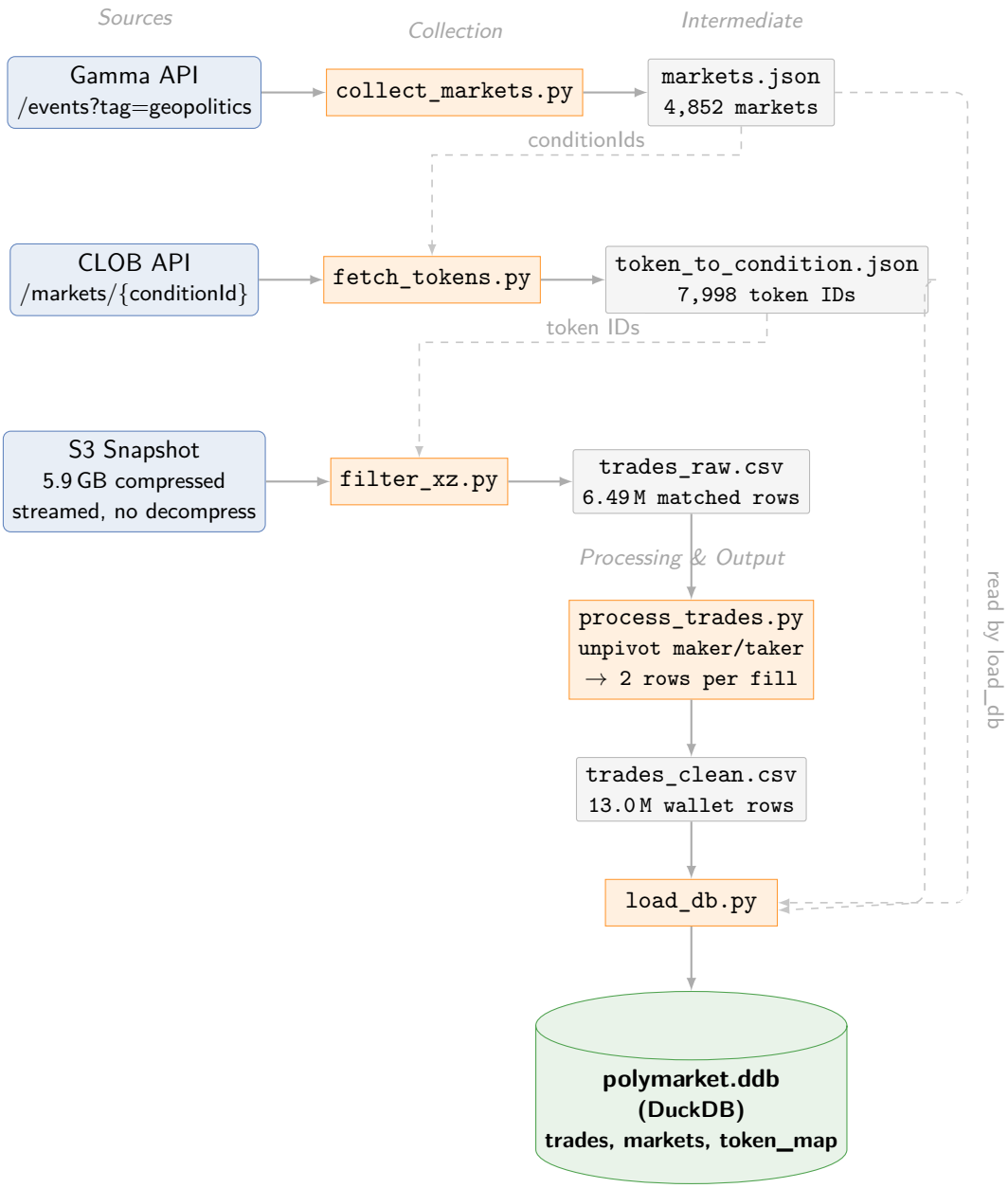

Figure 1: Data collection and loading pipeline. Orange: Python scripts (`src/pipeline/`), blue: external data sources, grey: intermediate files, green: analytical database. Solid arrows show the primary data flow; dashed arrows show secondary dependencies (metadata lookups). `filter_xz.py` streams the S3 snapshot without full decompression, matching token IDs on the fly. Full pipeline reproducible via `make pipeline`.

Step 0: S3 Snapshot Download:

The first step is downloading `orderFilled_complete.csv.xz` into `data/raw/` from the `warproxxx/poly_data` S3 archive (see Appendix A).

Table 3: Example rows from `orderFilled_complete.csv.xz`. Each trade assets are identified by ERC-1155 token IDs, these are 77-digit integers assigned at market creation, one for both Yes and No.`0` denotes USDC. No condition IDs or market names are present in the raw data, therefore the following steps of the pipeline exist to resolve this problem.

timestamp	maker	makerAssetId	makerAmt	takerAssetId	takerAmt
1675247746	`0x78db...`	`0` (USDC)	29.68	`55628...`	30.60
1675247746	`0xc6fd...`	`41572...`	30.60	`0` (USDC)	0.92

Step 1: `collect_markets.py`

The pipeline begins by querying the Gamma API for all closed geopolitical markets, paginating until exhausted, and writes the result to `data/raw/markets.json`. This file serves as the master market registry all over steps.

Listing 1: Gamma API query parameters.

```
r = httpx.get(f"{BASE}/events", params={ "limit": limit,
                                         "closed": "true",
                                         "tag_slug": "geopolitics",
                                         "offset": offset }) #
                                            filters: closed markets
                                             only, tagged with "
                                            geopolitics"
```

`resolvedOutcome` is not returned directly by the API, it is derived by parsing the `outcomePrices`, string into a list and selecting the outcome where the price goes to $1 at settlement. Over 5000 markets were retrieved with these parameters as of April 2026. The following table summarises the markets.json schema:

Table 4: `markets.json` field reference.

Field	Type	Example
`conditionId`	string	`0x0f99...`
`question`	string	Will Putin remain President through 2024?
`eventTitle`	string	Will Putin remain President through 2024?
`startDate`	ISO 8601	`2024-03-26T15:11:43Z`
`endDate`	ISO 8601	`2024-12-31T12:00:00Z`
`closedTime`	ISO 8601	`2025-01-01T09:42:22Z`
`volume`	float	`2,187,634`
`outcomes`	string[]	`["Yes", "No"]`
`outcomePrices`	string[]	`["1", "0"]`
`resolvedOutcome`	string	`Yes` (derived from `outcomePrices`)
`tags`	string[]	`["russia", "putin", "geopolitics", ...]`

Step 2 `fetch_tokens.py`

As shown in Table 3, snapshot rows carry no conditionID (cid), only ERC-1155 token IDs. `fetch_tokens.py` resolves this by querying the CLOB API for the YES/NO ERC-1155 token IDs of each market in `markets.json` with volume $\geq$\$1,000.

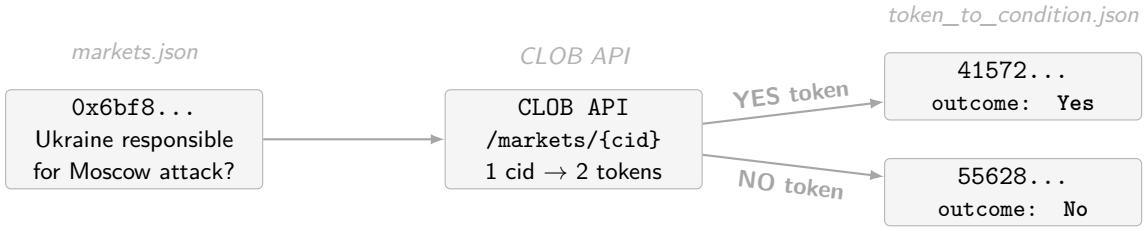

Figure 2: Each `conditionId` is resolved to a YES and NO ERC-1155 token ID via the CLOB API. The token IDs shown are real values from the dataset.

The \$1,000 volume filter excludes negligible markets, at time of writing, `token_to_condition.json` contained approximately 8,000 token IDs across 4,000 markets.

Step 3: `filter_xz.py`

Using the token to condition mapping, the `filter_xz.py` extracts matching trades from the full snapshot in a single streaming pass. Full decompression is impractical, given the archive size at 35-40 GB (Table 2). `xzcat`[2] (Unix utility) is used to stream the `.xz` file line by line into a Python CSV reader, without having to write the full decompressed file to disk.

Listing 2: Streaming the `.xz` archive without full decompression.

```
with subprocess.Popen(["xzcat", str(IN)],
                      stdout=subprocess.PIPE,
                      bufsize=1024*1024) as proc:
    reader = csv.reader(line.decode() for line in proc.stdout)
```

Each row is checked against `token_to_condition.json`: a match on either `makerAssetId` or `takerAssetId` means the row is written to the .csv file, with `condition_id` and `outcomes` appended to that output row.

Table 5: `trades_raw.csv` - output of `filter_xz.py`.

Property	Value
Rows	6,505,297 ($\approx$ 4.3% of full snapshot)
Unique wallets	333,124
File size	2.1 GB
Date range	1 February 2023 - 7 October 2025
Added columns	`condition_id`, `outcomes`

Step 4 `process_trades.py`

Each raw fill in `trades_raw.csv` records both the maker and taker in a single row. `process_trades.py` normalises this into wallet-centric rows suitable analysis:

- **Unpivot:** Each fill is split into two rows, a taker and a maker since every trade has exactly one buyer and seller. This expends the 6.5 M fills to 13.0 M wallet-centric rows.
- **Side inference:** For each `OrderFilled` event, the maker places the limit order and the taker fills it. When `makerAssetId` = 0 this indicates the maker paid USDC to acquire outcome tokens (`BUY`). Otherwise the maker delivered outcome tokens to receive USDC (`SELL`).
- **Normalisation:** raw integer amounts are divided by 10^6 to convert from ERC-1155 precision to USDC and token units
- **Price:** derived as `price` = `usd_amount`/`token_amount`, representing the implied probability at time of trade.

[2] https://linux.die.net/man/1/xzcat

- **Dust filter:** rows where the normalised token amount rounds to zero are discarded completely.

Table 6: Sample rows from `trades_clean.csv`. Each original fill becomes two rows with opposite sides.

timestamp	wallet	side	outcome	usd	tokens	price
2023-02-01	`0x78db...`	BUY	No	29.68	30.60	0.97
2023-02-01	`0x4bfb...`	SELL	No	29.68	30.60	0.97
2023-02-01	`0xc6fd...`	BUY	Yes	0.92	30.60	0.03
2023-02-01	`0x78db...`	SELL	Yes	0.92	30.60	0.03

Step 5: `load_db.py`

`load_db.py` is the final step of the core pipeline, loading the `trades_clean.csv` and `markets.json` into a single DuckDB analytical database (`polymarket.ddb`). Unix timestamps are converted into native `TIMESTAMP` columns, the `tags` field is converted from a raw JSON string to a native `VARCHAR[]` array. The token map is kept as a `token_map` table for possible future cross-reference. The resulting schema, and the database choice of DuckDB, are covered in Section 0.3.

0.3 Database Schema

All pipeline outputs are merged into a single DuckDB database (`polymarket.ddb`). DuckDB was chosen for three main reasons [Raasveldt and Mühleisen(2019)]:

- **Columnar execution:** its vectorised engine handles time-series aggregations and `GROUP BY` queries more efficiently than a row-oriented store such as SQLite, especially across 13 million wallet rows.
- **Native array support:** the `VARCHAR[]` type allows market tags to be stored and filtered directly, removing the need for a separate junction table.
- **Serverless:** like SQLite, no background process is needed, keeping the database fully portable as a single file within the project repository.

The database contains five tables, summarised in Table 7. Three are loaded by the core pipeline (`load_db.py`): `trades`, `markets`, and `token_map`. Two further tables, `sentiment` and `financial` are both populated by the sentiment pipeline (Section **??**) and joined to the core tables by the shared foreign key, `condition_id`.

Table 7: Database tables after the full pipeline.

Table	Rows	Loaded by
`trades`	13,010,594	`load_db.py`
`markets`	5,372	`load_db.py`
`token_map`	8,794	`load_db.py`
`sentiment`	—	sentiment pipeline (Section **??**)
`financial`	—	sentiment pipeline (Section **??**)

Figure 3 shows the full schema and relationships between tables. `condition_id` is the shared foreign key that links trade activity, market metadata, token identifiers, and the sentiment and financial enrichment layers.

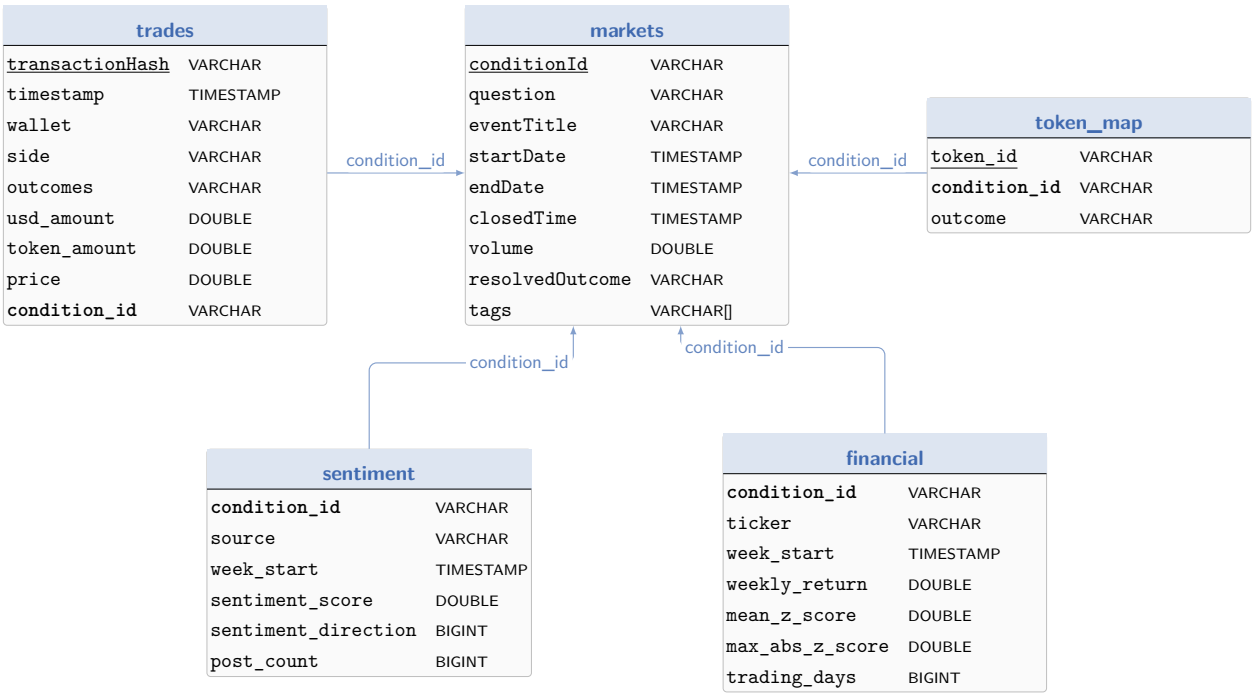

Figure 3: Database schema. Underlined fields denote primary keys; bold fields are foreign keys referencing `markets.conditionId`. The `sentiment` and `financial` tables are populated by the sentiment pipeline (Section **??**).

An exploratory analysis of the resulting dataset, covering wallet activity distributions, volume concentration, hit rates, and market coverage-is provided in Section **??**.

References

[Polymarket(2025)] Polymarket. Polymarket developer documentation, 2025. URL https://docs.polymarket.com.

[Raasveldt and Mühleisen(2019)] Mark Raasveldt and Hannes Mühleisen. Duckdb: an embeddable analytical database. In *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, pages 1981–1984, 2019.

[warproxxx(2025)] warproxxx. poly_data: Polymarket data retriever, 2025. URL https://github.com/warproxxx/poly_data.

A Data Download

The raw snapshot is retrieved by running the following command from the project root:

```
curl -L -o data/raw/orderFilled_complete.csv.xz \
    https://polydata-archive.s3.us-east-1.amazonaws.com/orderFilled_complete.csv.xz
```

This downloads the 5.9 GB compressed archive directly from the `warproxxx/poly_data` S3 bucket. Full setup instructions are in the project README at johnstew324/polymarket-whale-watch.

B Pipeline Source Code

C Pipeline Source Code